

Lesson 7 Teacher's Script

Core Mechanics I: On the Move

VIDEO GAME CREATION

[0:00-0:05] Welcome Back

"Welcome back! Last week you hit the Level 1 MVP milestone - a full, winnable game loop. Quick show of hands - who showed their win screen to somebody at home?" [Pause, call on 1-2 volunteers]

"Today we start making that loop feel like an actual game instead of a static puzzle. First change: your enemies are about to start moving."

[0:05-0:20] New Unit: Core Mechanics

"You just finished the 'Programming Basics' stretch of the course - variables, loops, collision, state tracking. Starting today, we're in a new unit: Core Mechanics. Over the next five lessons, we'll add losing, scoring, a real UI, and multiple waves - by the end of Lesson 11, your game will be a complete, replayable arcade shooter."

"Today's piece specifically: enemies stop sitting still. Every one of them will now creep toward the player. That one change touches almost everything else we build from here."

[Timing checkpoint: wrap by 0:20]

[0:20-0:40] Movement That Doesn't Stop: Continuous Motion

"So far, everything in your game has moved in response to something - a key press, a spawn. Today's movement is different: it just keeps happening, every single frame, with nothing triggering it."

[Demo cue: switch to Godot, open a project with a working win condition from Lesson 6]

"Watch: I've added a few lines to the enemy's script that run automatically, every frame, forever - move down a little, check if it's gone too far. No key press, no collision, nothing triggers this. It just runs." [Run the scene, let enemies drift down]

"Notice I multiplied the movement by delta - the time since the last frame. Why do you think that matters?" [Pause for guesses] "Right - without it, a fast computer would move enemies faster than a slow one. Multiplying by delta keeps the speed the same no matter how fast the game is running. You've actually seen this pattern before, all the way back in Lesson 2's player movement - today's just the first time an enemy uses it too."

[Discussion question] "What's the actual difference between 'runs every frame' and 'runs when something happens'? Can you think of something else in your game, or in a game you've played, that has to run continuously rather than in response to an event?" (Examples to draw out: a timer counting down, a camera following the player, gravity.)

[Timing checkpoint: wrap by 0:40]

[0:40-1:00] The Hidden Bug: When "Destroyed" Doesn't Mean "Defeated"

"Here's the part of today that matters most. Once enemies can move, they can now leave the screen at the bottom without ever being shot. That means there are now TWO different reasons an enemy might stop existing: getting shot, or escaping. And escaping should never count as a win toward your goal."

"For most of you, this just means adding a new, separate check - no changes to what you already built. But if you're in GameMaker or 001 Game Creator, we need to talk about something that's been quietly wrong since last week, and today is when it actually breaks."

[Demo cue: switch to GameMaker, open a Lesson-6-complete project]

"Last week, your win-counter was wired to the Destroy event - a built-in event that fires whenever an enemy is destroyed, no matter why. That was totally correct last week, because the ONLY way an enemy could be destroyed was by being shot. But today we're adding a second way - escaping - and the Destroy event can't tell the difference. It'll fire, and quietly decrement your win-counter, for an enemy that should NOT have counted."

"This is a really common category of bug in real software, not just games: code that was completely correct when it was written becomes wrong later, not because it changed, but because something else around it changed. The fix isn't complicated once you see it: move the win-counter to the one event that actually, specifically means 'this enemy got shot' - the collision event - and leave the generic Destroy event out of win-counting entirely."

[Demo cue: show the fix - Collision event gets the decrement, Destroy event goes empty] "See the difference? Collision with a bullet is specific - it can only ever mean one thing. Destroyed is generic - it means 'stopped existing,' full stop, for any reason at all. Attach your important logic to the specific event, not the generic one."

"001 Game Creator has the exact same issue, with its 'Defeated / Destroyed' trigger instead of a Destroy event - your handout walks through the identical fix there. If you're in one of the other six engines, your Lesson 6 build already tied the win-counter to the specific hit-detection code, so you have nothing to fix - just new code to add. Worth noticing why that turned out to be lucky, or well-designed, depending how you look at it."

[Discussion question] "Can anyone think of a real-world example of this same idea - a rule that was fine until something else changed and it quietly started causing problems?" (Open-ended; encourage a few guesses even if imperfect - the pattern-recognition matters more than a perfect answer.)

[Timing checkpoint: wrap by 1:00]

[1:00-1:05] Today's Exercise & Handouts

"One more thing before you open your projects. Looking back at Lesson 6, we noticed the win-counter ended up living in a different place in almost every engine - the main scene here, the spawner there, one project's collision branch somewhere else. Only one of you, building in

Unity, had it living on its own dedicated object built just to hold that kind of information. That turned out to be the better habit, so we've updated the Lesson 6 materials to match it everywhere, and I want you to update your own project too."

"It's a quick change, not a rebuild: create one object whose only job is holding game-wide information like the win-counter, and have everything else report to it instead of touching that number directly. Your updated Lesson 6 handout for your engine now shows exactly what that looks like - use it as your reference. Do this before you start today's movement work, since today's bug-fix discussion is about the exact same question: which piece of code should own which job."

[Give students 3-5 minutes to make the change if they haven't already, or confirm it's already in place. Circulate - GameMaker and Scratch tend to have the most questions here, since the retrofit is least like anything else they've built.]

"Quick question before we move on: why bother with a separate object for this at all? Couldn't the win-counter just keep living on the spawner, or the player, like it did before?" [Pause for guesses]

"A few real reasons this is worth doing, not just tidiness for its own sake. First - anyone joining this project later, including future-you in three weeks, has exactly one place to look for shared information, instead of having to remember or rediscover which object happens to be holding it this time. Second - it means objects don't have to reach into each other's private variables to get work done; the Bullet doesn't need to know anything about how the Spawner is built, it just talks to the GameManager. Third, and this is the one that pays off soonest: next lesson we're adding lives, and the lesson after that, score. Both of those are exactly the same kind of shared, game-wide information as today's win-counter - and they already have an obvious home waiting for them."

[Demo cue, optional: pull up any one engine's GameManager object live and point out it currently holds exactly two things - point out that by Lesson 9 it'll hold four, and that's the design working as intended, not scope creep.]

"This pattern has a bunch of different names depending on who you ask - Game Manager, Singleton, Game State - but it's the same idea everywhere: one object, one job, holding information the rest of the game needs to share."

"For lab today, open your own project from last week - not a new one, and no new assets needed. Every engine's handout has two parts: adding movement and escape cleanup, and - for GameMaker and 001 Game Creator only - fixing last week's win-counter."

[Hand out engine-specific handouts]

"GameMaker and 001 Game Creator folks - read the setup section of your handout carefully before touching anything. You're deleting code today, not just adding it."

"Questions before we start lab? ... Let's get these enemies moving."

Lab Time Notes

- The single most important thing to verify this week, for every student regardless of engine: let an enemy escape off the bottom unshot, and confirm "You Win!" does NOT appear. This is the real test of Level 2, not just watching enemies move.
- GameMaker / 001 Game Creator: watch specifically for double-counting - a student who added the new win-counter code but forgot to remove the old generic-hook code will see

the win message trigger after only about half the enemies are shot. This is the most likely mistake this week; check for it directly rather than waiting for a student to notice.

- A student whose enemies never disappear at the bottom likely has an off-screen threshold that doesn't match their project's actual screen size - have them adjust the number rather than guess at a fix.
- Circulate by engine cluster as usual. GameMaker and 001 Game Creator clusters need the most attention this week.
- Students who finish Level 2 early should be pointed at the Level 3 ideas on their handout, not given new unrelated work.